1. Introduction and Overview
2. Ad Click Prediction
3. Google Search
4. ChatGPT / Personal Assistant / Siri
5. Image Captioning
6. Machine Translation
7. High Relevance Image Search
8. Text to Image Generation
9. Personalized Headshot Generation
10. Text-to-Video Generation

10 Personalized Headshot Generation

Introduction

Personalized text-to-image (T2I) models are one of the emerging applications of generative AI. Imagine asking a T2I model to generate a photo of "a dog playing with a ball in a park" and receiving a photo of your own dog, not just any dog. This is possible by providing a set of reference photos showing it actively and clearly. In contrast, the same prompt without the reference photos would yield a generic image of a dog playing with a ball, such as a golden retriever. The dog, we want to personalize, is the subject, represented by a token like "sks" in the prompt, for example, "a photo of sks dog playing with a ball in a park".

Figure 1: Personalized T2I model generating personalized photos of a specific dog based on a few reference photos.

In this chapter, we explore how to develop a personalized T2I model capable of generating professional headshots of specific individuals, such as employees.

Clarifying Requirements

Here is a typical interaction between a candidate and an interviewer.

Candidate: Are the generated headshots primarily intended for business profiles, such as LinkedIn?

Interviewer: Yes, exactly.

Candidate: Lawsuits users will provide several images of themselves to different vendors. Jony eagle - with their faces visible, and the model will learn their appearance. Do we have the rights to use these photos for training?

Interviewer: Assume we have the rights to use the uploaded photos for training purposes.

Candidate: What if some images are not suitable, like if they're too dark or the face isn't visible?

Interviewer: We need to remove those and prefer the user to provide better images.

Candidate: Should users be able to specify features such as hairstyle in the generated images?

Interviewer: For simplicity, the initial version won't support

Candidate: What resolution is expected for the headshots?

Interviewer: The system should support 512x512 resolution.

Candidate: Can anyone use an client-front's personalized generator, TO model?

Interviewer: No.

Candidate: Should users be able to upload photos to control the generated headshots?

Interviewer: We prefer to keep it simple, so assume users can't provide their prompts.

Candidate: How many headshots images should a user expect?

Interviewer: 50 images.

Candidate: What is the expected latency?

Interviewer: Training can take hours, but it's fine if it's not. Inference should take no more than 10 minutes.

Frame the Problem as an ML Task

In this section, we can hypothesize a generative AI to a case study to explore an important aspect of image generation personalization. The process involves adapting a pretrained T2I model to learn a new subject. In this case, the new subject.

Specifying the system's input and output

The input includes several images of the user's face, taken from different angles and at different poses. The output consists of professional headshots in the desired style. The system includes an high quality and diverse set this process involves the process.

Figure 2: Input and output for a personalized headshot generation system

Choosing a suitable ML approach

Diffusion models excel consistently and of generating very detailed and realistic images. A common practice for personalization is to start with a T2I model trained on a large dataset, then fine-tune it on the specific subject.

There are three approaches to personalizing a pretrained T2I model: furthermore fine-tuning based and tuning-free.

Tuning-based methods fine-tune the model to learn a set of reference images and the subject identifier. This approach integrates the new identity into the model, allowing it to generate a variety of images while preserving the identity.

Figure 3. Fine-tuning-based approach for T2I personalization

Tuning-free methods, such as Dreambooth, learn a "token" or embedding, which represents the subject. This token can then be used in prompts to generate images featuring the subject in various scenes and styles. This method offers more flexibility and efficiency compared to fine-tuning, allowing for quick personalization without extensive retraining.

Figure 4. Tuning-free approach for T2I personalization

Fine-tuning offers higher quality and better control over the model's behavior, making it suitable for applications where high fidelity is crucial. However, it requires more computational resources and time. Tuning-free methods, on the other hand, prioritize speed and efficiency, making them suitable for applications where quick personalization is important and resources are limited.

We choose a tuning-based approach for this system, as it prioritizes high-quality and diverse outputs, which aligns with the desired characteristics of professional headshots.

Choosing a suitable ML approach

Diffusion models consist consistently and of generating very detailed and realistic images. A common practice for personalization is to start with a T2I model trained on a large dataset, then fine-tune it on the specific subject.

There are three approaches to personalizing a pretrained T2I model: fine-tuning-based and tuning-free.

Tuning-based methods fine-tune the model to learn a set of reference images and the subject identifier. This approach integrates the new identity into the model, allowing it to generate a variety of images while preserving the identity.

Figure 5. Fine-tuning-based approach for T2I personalization

On the other hand, tuning-free methods bypass the need for fine-tuning the T2I model for each new identity. Instead, they freeze the pretrained T2I model and employ a text encoder, text. Rather than fine-tuning the model, these methods leverage a text encoder to map reference image and objects from the input. This allows the model to generate personalized images without adjusting its internal weights for each identity.

Figure 6. Tuning-free approach for T2I personalization

Fine-tuning, tuning-based methods tend to capture more detailed features of the subject. They are also more versatile, handling a wider range of subjects and styles more effectively. Tuning-free methods, on the other hand, focus on tuning-based methods in the rest of this chapter. If you are interested in learning more about tuning-free methods, refer to DreamBooth.

Tuning-based methods can be used to achieve personalization, each offering unique advantages and disadvantages. Several techniques exist, including:

- Textual Inversion
- DreamBooth
- Low-rank adaptation (LoRA)

Textual Inversion

Textual inversion (TI) personalizes a T2I model by introducing a new special token that represents the subject and learning its embedding. The process involves adding the special token's embedding to the text encoder, keeping the diffusion model, and exciting other model embeddings unchanged.

Figure 7. Textual Inversion for personalizing a pretrained T2I model

Initially, a random embedding is assigned to the new token. The model then learns to adjust this embedding to accurately represent the subject in the generated images. The training process involves feeding the model with images of the subject and optimizing the embedding to minimize the difference between the generated images and the reference images.

Textual Inversion offers a lightweight approach to personalization, requiring only a few images and a short training time. It preserves the model's original capabilities and avoids modifying the model's weights, ensuring that the model retains its ability to generate a wide variety of images. However, it may not be as effective for capturing complex or highly detailed features compared to other methods.

DreamBooth

DreamBooth (D) is a popular personalization method that introduces by Google in 2023. It features a pretrained diffusion model using a unique of the subject of interest. Unlike textual inversion, DreamBooth updates all the diffusion model's parameters during fine-tuning. This allows the model to capture more subtle details and nuances.

Figure 8. DreamBooth method updating the entire diffusion model

DreamBooth differs primarily in its methodology for customized Pretraining:

- Rare token identifier
- Class-specific prior preservation loss

Rare token identifier

Most personalization methods select an arbitrary token to represent a different subject. While textual inversion creates a new token, DreamBooth leverages existing tokens in the vocabulary, ensuring the pretraining contains little information about these tokens. Creating and training token can't work well in practice, Let's explore why simple approaches such as selecting a common or random identifier can't work, and why rare tokens are the most suitable choice.

Pretraining on a common token identifier

A simple way to represent a subject of interest is to choose a common English word such as "unique" or "special". This approach is problematic because the token usually has an established meaning. For example, if we choose "special" to refer to the subject represented by "sks", the model might interpret "sks" as a general term for something special, rather than the specific individual intended for the number of language. The model might struggle to learn how to associate the token with the subject, hindering its ability to generate relevant images.

Another way to represent the subject is to randomly combine characters. This approach also poses issues because tokenizer might break each character sequence leading to strong prior assumptions for these characters. For example, if we choose "sks" as the identifier, the tokenizer might split it into individual characters, such as "s", "k", and "s". The model might have strong associations within the vocabulary. This misinformation can cause the model to generate outputs that are influenced by the meanings or patterns associated with those individual characters or sub-units, rather than treating the identifier as a unique and independent entity.

Pretraining on a rare token identifier

DreamBooth addresses these challenges by selecting rare tokens—those that appear infrequently in the training data. Representing the subject with these rare tokens allows the model to learn a distinct representation for the subject, minimizing interference with existing knowledge and associations. This approach ensures that the model can effectively learn and generate images of the subject without disrupting its understanding of other concepts.

Class-specific prior preservation loss

DreamBooth fine-tunes all layers of the diffusion model. While this enhances the quality of generated images, it can lead to overfitting, where the model loses diversity in its outputs. For example, fine-tuning the model on images of a specific dog breed might cause it to struggle to generate images of other dogs.

To address this, DreamBooth introduces a class-specific prior preservation loss to maintain general class characteristics. This encourages the model from overfitting to the specific dataset and helps it to generate diverse images that belong to the broader class. We will examine DreamBooth's loss function in the training section.

DreamBooth has several prior terms:

- **Effective of learning subject details:** Updating more parameters allows the model to learn the details of the subject more accurately.
- **Prevent Class Impact:** Because the entire diffusion model is updated, a smaller image set is needed to learn the subject.

Cons:

- **Disadvantage requirements:** After fine-tuning for each subject, the entire diffusion model has to be stored for future use. This can require several gigabytes of storage for each subject, which is costly and impractical.
- **Resource intensive:** Updating the entire diffusion model demands more GPU memory during training.

In contrast, DreamBooth learns subject details effectively but is costly to train and store. In contrast, textual inversion is efficient and compact, but less effective. Next, we'll explore LoRA, which offers a balanced approach.

LoRA

LoRA, introduced by Microsoft (in 2021), is a powerful method for efficiently fine-tuning large models. This method was originally designed for adapting large language models (LLMs) to specific tasks but was later adapted for other tasks including text-to-image models.

The key motivation behind LoRA is that fine-tuning all parameters of large pretrained models, such as GPT-3, is time-consuming and costly. Instead, LoRA suggests that only a small subset of parameters need to be updated to achieve good performance and updating only this subset can significantly reduce computational resources and time.

Due to the importance, we'll explore LoRA in detail in the training section.

The mathematics of LoRA

Let's consider a pretrained weight matrix $W \in \mathbb{R}^{d \times k}$, transforms the input vectors $x \in \mathbb{R}^k$, into the output vector $y = Wx$.

Figure 9. A fully connected neural network layer

The goal of Fine-tuning is to adjust weight parameters, W, to improve the performance of the specific task. Instead of modifying W directly, LoRA introduces a modification by contributing an additional low-rank component, ΔW, which can be represented as a product of two smaller matrices:

$$\Delta W = AB$$

Where:

- $A \in \mathbb{R}^{d \times r}$,
- $B \in \mathbb{R}^{r \times k}$,
- d, and k are the original dimensions and
- r is a small integer representing the rank, typically much smaller than d and k.

Figure 10. LoRA method training and learning new parameters in the diffusion model

Pros:

- **Resource efficiency:** Significantly reduces GPU memory and storage requirements by freezing the original model parameters and only training the smaller matrices. This makes LoRA updates only a small fraction of the model's parameters, making it more efficient than full-rank updates.
- **Minimizes storage requirements:** Stores the original model remains unchanged, only the LoRA layers are stored. This significantly reduces storage requirements compared to storing a separate fine-tuned version for each subject.

Table 1 provides a comparison of the three tuning-based personalization methods.

	Textual Inversion	LoRA	DreamBooth
Learning effectiveness	Low	Moderate	High
Inference time	Low	Moderate	High
Required memory	Low	Moderate	High
Maintaining the original model's capabilities	Yes	Yes	No

Table 1. Comparison of popular tuning-based personalization methods.

Which method is more suitable for headshot generation?

The suitability of these methods depends on the use case and the system requirements. For headshot generation, we choose DreamBooth for the following reasons:

- **Better identity preservation:** DreamBooth is more effective at preserving details of the subject, leading to more identity-focused outputs.
- **Sufficient storage capacity:** Assuming ample storage, DreamBooth's increased storage requirements are a less significant drawback.
- **Infrequent inference:** Since fine-tuning is a one-time process, DreamBooth's longer inference time is less critical.

While LoRA offers a balanced approach, DreamBooth's focus on identity preservation aligns with the system's primary goal of generating high-quality headshots. The ability to capture subtle details and nuances is crucial for this application, and the increased storage and inference time are acceptable trade-offs.

Data Preparation

The number of images needed varies depending on the method. Tuning-free methods generally require just one image, whereas tuning-based methods such as DreamBooth need around 10–20 images.

We use tuning-based methods, such as LoRA and DreamBooth, in this chapter. These images may vary in different resolutions and aspect ratios. The system must first resize the images, as follows these steps:

- Image cropping
- Image standardization
- Generic face data addition

Figure 11. Preparing data for training

Image resizing

Diffusion models typically require fixed-size datasets, but user-uploaded images often vary in dimensions. We resize the images to uniform dimensions before feeding them into the diffusion model.

Image augmentation

Tuning-based methods rely on a limited set of reference images, which can lead to overfitting. To mitigate this, we can apply random transformations like cropping, rotation, and slight adjustments to brightness and contrast. These augmentations create variations in the training data, helping the model generalize better to unseen poses and lighting conditions.

Generic face data addition

To prevent the model from forgetting general image knowledge, we add a portion of generic face data, such as the FFHQ dataset, to the training data. This helps the model retain its ability to generate realistic images of faces in various poses and styles.

Model Development

Architecture

The DreamBooth method fine-tunes a pretrained diffusion model. We use a model with a U-Net architecture, pretrained to output random images from the LAION-5B dataset (mentioned in Chapter 9). The architecture leverages a series of downsampling blocks followed by a series of upsampling blocks.

Training

To fine-tune a pretrained diffusion model, we follow the same process as training a diffusion model from scratch.

- **Noise addition:** Noise is added to clean images in a randomly selected timestep.
- **Conditioning signal preparation:** Repeated tokenizer prepares the image caption and timestep as conditioning signal for the diffusion model.
- **Noise prediction:** The model predicts the noise to be removed from the noisy image using the conditioning signal.

The training steps consists of: original images are first resized, then cropped, random horizontal flips are applied, and finally normalized to pixel values ranging from -1 to 1. The tokenizer processes the corresponding captions, converting them into numerical representations suitable for the model. The noise scheduler adds random noise to the processed images, creating noisy versions at various timesteps. The model then predicts the noise added to the images, using the noisy versions and the tokenized captions as input. The difference between the predicted noise and the actual noise added is used to calculate the loss, which guides the model's learning process.

Figure 12. Training process for DreamBooth

Loss function

The training process for DreamBooth aims to minimize the difference between the predicted noise and the actual noise added to the images. This difference is quantified by a loss function, which guides the model's learning process. The loss function for DreamBooth combines two terms:

- **Reconstruction loss:** Measures the difference between the predicted noise and the actual noise added to the images.
- **Class-specific prior preservation loss:** Encourages the model to generate diverse images that belong to the broader class of the subject, preventing overfitting to the specific dataset.

The overall loss function is a weighted combination of the reconstruction loss and the class-specific prior preservation loss. The formula for the overall loss is:

$$\text{DreamBooth Loss} = \text{reconstruction loss} + \lambda \times \text{class-specific prior preservation loss}$$

λ and σ are hyperparameters that control the trade-off between maintaining a specific identity and preserving human-specific characteristics. The reconstruction loss ensures the model learns the subject's identity, while the class-specific prior preservation loss prevents overfitting and maintains the model's ability to generate diverse images within the broader class.

Figure 13. Overall loss calculation

Sampling

The sampling process for headshot generation is similar to the T2I generation discussed in Chapter 9, as both use diffusion models. However, it has differences due to a few unique feature prompts. In Chapter 9, the user provided the text prompt. For headshot generation, the system uses a predefined prompt template, incorporating the identifier and the class noun.

In headshot generation, the users don't provide prompts; the system creates one at the backend. The prompt template typically represents various professional settings and includes the identifier used during training to ensure the generated images reflect the user's identity. Some examples include:

- "A professional headshot of [V] photo in front of a plain white background"
- "A photo of [V] in a suit and tie, professional lighting"
- "A headshot of [V] with soft lighting, looking slightly to the left"
- "A photo of [V] against a neutral outdoor background"

The identifier, represented by "[V]", ensures the model generates images of the specific individual.

After crafting the prompts, we sample over images and prompts from the diffusion model. We follow the standard diffusion process sampling steps:

1. Random noise initialization
2. Iterative denoising: The model predicts noise, using classifier-free guidance (CFG) (Chapter 9) during each step to reduce noise and refine image quality. The formula for CFG is: $\text{Output} = \text{Unconditional} + \text{Guidance Scale} \times (\text{Conditional} - \text{Unconditional})$

Figure 14. Sampling a headshot image

Evaluation

Offline evaluation metrics

It is important to evaluate personalized diffusion models to ensure our model is capable of preserving the user's identity in the generated images. We assess the performance of personalized diffusion models by focusing on three main aspects:

1. Identity preservation
2. Prompt adherence
3. Image alignment

Text alignment

Text alignment refers to how closely generated images match text prompts. A common metric for measuring this is the CLIP score, which ensures the images are not only high quality but also relevant to the input text.

Image quality

We use inception score (IS) to measure the quality and diversity of generated images. This metric assesses the visual clarity and variety of the images. A higher IS indicates better image quality and greater diversity, suggesting the model can generate a wide range of visually appealing and distinct images.

Image alignment

Image alignment evaluates how closely generated images resemble the reference images. This ensures the model accurately captures the subject's identity and the nuances of their appearance. We use DINO score to measure this. DINO leverages self-supervised learning to learn robust representations of images, making it sensitive to subtle differences in visual features. This sensitivity makes it ideal for evaluating image alignment, as it can effectively capture the subtle differences between generated and reference images.

Evaluation

Offline evaluation metrics

It is important to evaluate personalized diffusion models to ensure our model is capable of preserving the user's identity in the generated images. We assess the performance of personalized diffusion models by focusing on three main aspects:

1. Identity preservation
2. Prompt adherence
3. Image alignment

Text alignment

Text alignment refers to how closely generated images match text prompts. A common metric for measuring this is the CLIP score, which ensures the images are not only high quality but also relevant to the input text.

Image quality

We use inception score (IS) to measure the quality and diversity of generated images. This metric assesses the visual clarity and variety of the images. A higher IS indicates better image quality and greater diversity, suggesting the model can generate a wide range of visually appealing and distinct images.

Image alignment

Image alignment evaluates how closely generated images resemble the reference images. This ensures the model accurately captures the subject's identity and the nuances of their appearance. We use DINO score to measure this. DINO leverages self-supervised learning to learn robust representations of images, making it sensitive to subtle differences in visual features. This sensitivity makes it ideal for evaluating image alignment, as it can effectively capture the subtle differences between generated and reference images.

Commonly used metrics for measuring the visual similarity between the generated and original subject are:

- CLIP score
- DINO score
- Facial similarity score

CLIP score

CLIP model [12] uses two encoders—one for images and one for text. These encoders are trained to ensure that the image and text embeddings of a relevant image-text pair are close to one another in the embedding space.

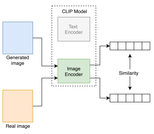

Figure 14: Image alignment calculation with CLIP

To measure image alignment using CLIP, we discard the text encoder and use the image encoder to generate embeddings for both the generated and real images. We then calculate the cosine similarity between these embeddings to assess their similarity. Higher scores indicate that the personalized diffusion model creates images that are more visually similar to real images.

DINO score

DINO [13] is a self-supervised learning method developed by Meta. DINO learns visual representations of images without the need for labeled data. In particular, it uses a method called contrastive learning [14], whereby the model learns to distinguish between similar and dissimilar images by organizing them in an embedding space—similar images are placed closer together, and dissimilar ones are positioned farther apart.

DINO, along with its more recent variations like DINOv2 [15], is particularly good at capturing similarities between images because it is trained to recognize subtle differences. This makes DINO particularly effective for measuring image alignment. By comparing the embeddings of a generated image with a real one, DINO can evaluate how well the generated image matches the real one.

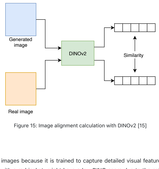

Figure 15: Image alignment calculation with DINOv2 [15]

DINO versus CLIP

DINO is preferred for comparing images because it is trained to capture detailed visual features. For example, two images, one with a yellow jacket and another with a red jacket, might have a low DINO score due to the color difference. On the other hand, CLIP is better for comparing images with text, as it is trained to match descriptions with visuals. The same images with different jacket colors might have a high CLIP score if both reflect a person wearing a jacket.

Facial similarity score

While CLIP and DINO measure visual similarity between generated and real images, they aren't designed to assess identity preservation. For example, two face images might show different individuals, but CLIP and DINO could still give a high similarity score. To address this, we use a face recognition model to compare generated and real images. These models specialize in identifying and measuring facial similarity, which is a crucial requirement in a personalized headshot generation system.

Combining DINO, CLIP, and facial similarity scores provides a comprehensive evaluation of image alignment in the personalized diffusion model.

Online evaluation metrics

Online evaluation is crucial in headshot generation. It measures user satisfaction directly, which is vital when users pay for the service, as higher satisfaction often leads to increased revenue. We focus on two primary metrics:

- **User feedback:** This metric directly reflects user satisfaction. After receiving their generated headshots, users rate their satisfaction on a scale from 1 to 5. High ratings indicate that the headshots meet or exceed expectations, while lower scores highlight improvements are needed.
- **Conversion rate to paid service:** This metric measures the percentage of users who move from showing interest to becoming new paying customers. It is calculated by dividing the number of new paying customers by the total number of users who engaged with the service—such as visiting the website, signing up for a trial, or making inquiries—within a specific time period.

Overall ML System Design

Generating professional headshots requires more than just a diffusion model. In this section, we examine three key pipelines:

- Data pipeline
- Training pipeline
- Inference pipeline

Data pipeline

This pipeline has two responsibilities:

- Preparing images with the subject of interest
- Preparing images of generic human faces

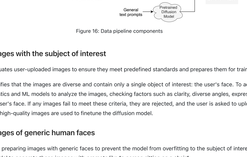

Figure 16: Data pipeline components

Preparing images with the subject of interest

This process evaluates user-uploaded images to ensure they meet predefined standards and prepares them for training.

Specifically, it verifies that the images are diverse and contain only a single object of interest: the user's face. To achieve this, we use various heuristics and ML models to analyze the images, checking factors such as clarity, diverse angles, expressions, and the presence of the user's face. If any images fail to meet these criteria, they are rejected, and the user is asked to upload more. This ensures that only high-quality images are used to fine-tune the diffusion model.

Preparing images of generic human faces

This step involves preparing images with generic faces to prevent the model from overfitting to the subject of interest. We use the pretrained T2I model to generate these images with prompts like "a person sitting on a chair."

Training pipeline

This pipeline is responsible for personalizing the pretrained diffusion model.

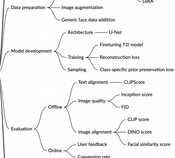

Figure 17: Fine-tuning a pretrained diffusion model

Inference pipeline

The inference pipeline is responsible for generating the headshots of the user using a personalized T2I model. Three main components in the inference pipeline are:

- Image generator
- Quality assessment service
- Uploader service

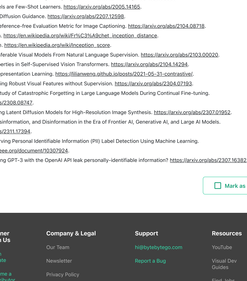

Figure 18: Inference pipeline components

Image generator

The image generator utilizes the personalized T2I model and hand-engineered text prompts to generate one image per prompt.

Quality assessment service

This service ensures the generated images meet identity preservation standards. This service uses a pretrained face recognition model to compare the generated headshot with the user's real images. If the generated image fails to preserve the user's identity, the service rejects it and requests that the image generator produce a new image using the same text prompt but with a different initial noise.

Uploader service

The uploader service manages the delivery of generated images to the user. It uploads the images to cloud storage so users can download their headshots.

Other Talking Points

If there is extra time at the end of the interview, here are some additional talking points:

- Preventing catastrophic forgetting during the fine-tuning [16].
- The details of choosing a rare token for the subject, and its importance [5].
- Details of class-specific prior preservation loss [5].
- Addressing the issue of reduced output diversity after fine-tuning [5].
- Supporting multiple sizes and aspect ratios in generated images [17].
- Details of tuning-free methods such as Meta's Imagine Yourself [1].
- Mitigating the risks and ethical concerns around deepfake generation and detection [18].
- ML techniques to handle personally identifiable information (PII) securely while ensuring data privacy [19][20].

Summary

Reference Material

[1] Imagine yourself: Tuning-Free Personalized Image Generation. https://ai.meta.com/research/publications/imagine-yourself-tuning-free-personalized-image-generation/

[2] MoA: Mixture-of-Attention for Subject-Context Disentanglement in Personalized Image Generation. https://arxiv.org/abs/2404.11565.

[3] InstantID: Zero-shot Identity-Preserving Generation in Seconds. https://arxiv.org/abs/2401.07519.

[4] An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion. https://textual-inversion.github.io/.

[5] DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation. https://arxiv.org/abs/2208.12242.

[6] LoRA: Low-Rank Adaptation of Large Language Models. https://arxiv.org/abs/2106.09685.

[7] Language Models are Few-Shot Learners. https://arxiv.org/abs/2005.14165.

[8] Classifier-Free Diffusion Guidance. https://arxiv.org/abs/2207.12598.

[9] CLIPScore: A Reference-free Evaluation Metric for Image Captioning. https://arxiv.org/abs/2104.08718.

[10] FID calculation. https://en.wikipedia.org/wiki/Fréchet_inception_distance.

[11] Inception score. https://en.wikipedia.org/wiki/Inception_score.

[12] Learning Transferable Visual Models From Natural Language Supervision. https://arxiv.org/abs/2103.00020.

[13] Emerging Properties in Self-Supervised Vision Transformers. https://arxiv.org/abs/2104.14294.

[14] Contrastive Representation Learning. https://lilianweng.github.io/posts/2021-05-31-contrastive/.

[15] DINOv2: Learning Robust Visual Features without Supervision. https://arxiv.org/abs/2304.07193.

[16] An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning. https://arxiv.org/abs/2308.08747.

[17] SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis. https://arxiv.org/abs/2307.01952.

[18] Deepfakes, Misinformation, and Disinformation in the Era of Frontier AI, Generative AI, and Large AI Models. https://arxiv.org/abs/2311.17394.

[19] Privacy-Preserving Personal Identifiable Information (PII) Label Detection Using Machine Learning. https://ieeexplore.ieee.org/document/10303924.

[20] Does fine-tuning GPT-3 with the OpenAI API leak personally-identifiable information? https://arxiv.org/abs/2307.16382.

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
help@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev. Guides
Find Jobs
Prepare for Coding Interviews

Copyright ©2022-2026 Bytebytego Inc. All rights reserved.

Ask AI